

Nghiên cứu về một lớp bài toán quy hoạch động

Từ Nguyên Thịnh
Trường Trung học số 1 Trường Sa, Hồ Nam

2009

Nguồn gốc: A study of a class of dynamic programming problems

I0I-China-Candidate-Team-Papers/2009/Xu-Yuansheng/future-cost-dp.pdf

Abstract

Bài viết khảo sát, thông qua bốn bài toán, một lớp bài toán quy hoạch động khá đặc biệt: quyết định ở hiện tại ảnh hưởng đến chi phí của “hành động” trong tương lai. Nếu ảnh hưởng của quyết định hiện tại lên tương lai chỉ phụ thuộc vào chính quyết định hiện tại, ta trực tiếp tính ảnh hưởng đó vào chi phí của quyết định hiện tại rồi truyền qua trạng thái. Nếu ảnh hưởng lên tương lai còn phụ thuộc vào tình huống tương lai, ta bổ sung trạng thái để giả định tình huống tương lai; khi đi đến quyết định tương lai thì dùng trạng thái đã giả định đó. Đây là phương pháp giải được trình bày chi tiết trong bài.

Từ khóa: quy hoạch động, tính trước chi phí, giả định quyết định tương lai.

1 Mở đầu

Trong các bài toán quy hoạch động thông thường, chi phí do “hành động” ở trạng thái hiện tại thường được tính đồng thời với trạng thái đó. Chẳng hạn trong bài toán gộp đá kinh điển, phương trình là

$$f[i][j] = \max_k \{f[i][k] + f[k+1][j]\} + w(i, j).$$

Khi tính $f[i][j]$, ta mới cộng chi phí của hành động gộp toàn bộ đồng đá từ i đến j . Điều này rất phù hợp với thói quen tư duy.

Tuy nhiên những năm gần đây thường xuất hiện một lớp bài toán quy hoạch động trong đó một phần chi phí của “hành động” hiện tại phải được tính từ các quyết định trước đó, rồi thông qua trạng thái mà tác động đến trạng thái hiện tại. Nguyên nhân của cách tính đặc biệt này là quyết định hiện tại có ảnh hưởng đến chi phí của “hành động” trong tương lai. Việc xây dựng phương trình cho lớp bài toán này thường khó hơn, cần phân tích kỹ đề bài và tìm ra mâu thuẫn cốt lõi.

2 Quyết định hiện tại chỉ ảnh hưởng đến tương lai qua chính nó

Xét một ví dụ đơn giản: trong hai nam và hai nữ cần chọn một nam một nữ đi làm nhiệm vụ, biết hiệu suất của từng người và muốn tổng hiệu suất lớn nhất. Nếu nam A được chọn thì hiệu suất của mọi nữ tăng 7; nếu nam B được chọn thì hiệu suất của mọi nữ giảm 7. Ở giai đoạn thứ nhất, chọn A hay B có ảnh hưởng khác nhau đến hiệu suất của nữ ở giai đoạn thứ hai. Ta có thể xem ảnh hưởng này là “sức hút cá nhân” của nam: cộng 7 vào hiệu suất của A và trừ 7 khỏi hiệu suất của B . Thực chất, ta đã tính chi phí của giai đoạn thứ hai ngay ở giai đoạn thứ nhất. Với dạng bài như vậy, ta thường gộp ảnh hưởng lên “hành động” tương lai vào chi phí của quyết định hiện tại. Hai ví dụ sau trình bày chi tiết hơn ý tưởng này.

2.1 Bài toán 1: Những quả cầu của Sue (SDTSC 2008)

Đề bài

Sue và Sandy gần đây say mê một trò chơi máy tính. Câu chuyện trong trò chơi diễn ra trên biển cả đẹp, bí ẩn và đầy kích thích. Sue có một chiếc thuyền nhỏ nhẹ, nhưng mục tiêu của Sue không phải làm hải tặc, mà là thu thập những quả trứng màu đang trôi nổi trong không trung. Sue có một vũ khí bí mật: chỉ cần chèo thuyền đến đúng phía dưới một quả trứng rồi dùng vũ khí đó, cô có thể thu thập quả trứng ngay lập tức.

Mỗi quả trứng có một giá trị hấp dẫn, và giá trị này giảm dần theo thời gian rơi trong không trung. Muốn được nhiều điểm hơn, Sue phải cố thu thập quả trứng khi giá trị hấp dẫn còn cao. Nếu một quả trứng rơi xuống biển, giá trị hấp dẫn của nó trở thành một số âm, nhưng điều này không làm Sue mất hứng, vì mỗi quả trứng đều khác nhau và Sue muốn thu thập tất cả.

Sandy thì không lãng mạn như Sue; Sandy muốn đạt nhiều điểm nhất có thể. Để giải quyết bài toán, trước hết Sandy trừu tượng hóa trò chơi thành mô hình sau.

Lấy mặt phẳng ngang chứa vị trí ban đầu của Sue làm trục x . Ban đầu có N quả trứng trên không. Với quả trứng thứ i , vị trí ban đầu là tọa độ nguyên (x_i, y_i) . Sau khi trò chơi bắt đầu, nó rơi đều theo chiều âm của trục y với vận tốc v_i đơn vị khoảng cách trên một đơn vị thời gian. Vị trí ban đầu của Sue là $(x_0, 0)$. Sue có thể di chuyển theo chiều dương hoặc âm của trục x , với vận tốc 1 đơn vị khoảng cách trên một đơn vị thời gian. Việc dùng vũ khí bí mật để lấy một quả trứng là tức thời; điểm nhận được bằng một phần nghìn tung độ hiện tại của quả trứng.

Hãy giúp Sue và Sandy: với điều kiện phải thu thập mọi quả trứng, hãy làm cho tổng điểm lớn nhất.

Dữ liệu vào

Dòng đầu gồm hai số nguyên N, x_0 , cách nhau bởi một dấu cách, lần lượt là số quả trứng và vị trí ban đầu của Sue. Dòng thứ hai gồm N số nguyên x_i . Dòng thứ ba gồm N số nguyên y_i . Dòng thứ tư gồm N số nguyên v_i , là vận tốc rơi đều theo chiều âm của trục y .

Dữ liệu ra

In một số thực, làm tròn đến ba chữ số sau dấu thập phân: điểm lớn nhất có thể đạt được khi đã thu thập tất cả quả trứng.

Ví dụ

```
ball.in
3 0
-4 -2 2
22 30 26
1 9 8
```

```
ball.out
0.000
```

Ràng buộc

Với 30% dữ liệu, $N \leq 20$. Với 100% dữ liệu, $N \leq 1000$ và $-10^4 \leq x_i, y_i, v_i \leq 10^4$.

Phân tích

Trước hết sắp xếp tất cả điểm, bao gồm cả điểm xuất phát, theo hoành độ. Khi đó bài toán trở thành: xuất phát từ điểm ban đầu, mỗi lần đi sang trái hoặc sang phải để đến một quả trứng gần nhất chưa lấy và bắn rơi nó. Gọi t_i là thời điểm lần đầu tiên đi đến quả trứng thứ i , đáp án cần tối đa hóa là

$$\sum_i (y_i - t_i v_i).$$

Ta nhận thấy tập các quả trứng đã bắn rơi luôn mở rộng thành một đoạn. Vì thế rất tự nhiên nghĩ đến $f_1[i][j]$ và $f_2[i][j]$: lần lượt là điểm lớn nhất khi đã bắn rơi đoạn quả trứng từ i đến j , hiện đang dừng ở điểm i hoặc điểm j . Xét $f_1[i][j]$, tức điểm i là quả trứng vừa bắn. Điểm nhận được khi bắn phụ thuộc vào thời điểm hiện tại, nhưng thời điểm này không thể biểu diễn trong trạng thái $f_1[i][j]$. Nếu thêm một chiều trạng thái để biểu diễn thời gian, độ phức tạp sẽ không chấp nhận được.

Mâu thuẫn ở đây là phương án di chuyển trong quá khứ ảnh hưởng đến điểm bắn ở hiện tại. Hãy xét kỹ hơn cách tính $f_1[i][j]$. Điểm bắn quả trứng i là $y_i - tv_i$, trong đó t bằng tổng thời gian di chuyển của mọi quyết định trước đó. Do đó, giống ví dụ chọn nam nữ ở trên, ta có thể tính phần $-tv_i$ ngay trong các bước di chuyển trước. Nói cách khác, mỗi lần di chuyển phải tính luôn lượng điểm chắc chắn sẽ mất trong tương lai. Chẳng hạn, khi chuyển từ $f_1[i][j]$ sang $f_1[i-1][j]$, tức đi từ i đến $i-1$, ngoài đoạn quả trứng i đến j đã bắn rơi, mọi quả trứng khác đều đang rơi; lượng điểm bị mất này được tính vào chi phí đi từ i đến $i-1$. Vì phần $-tv_i$ đã được tính ở các quyết định trước, lúc bắn chỉ cần cộng y_i .

Để mô tả thuận tiện, đặt

$$w[i][j] = \sum_{r=1}^n v_r - \sum_{k=i}^j v_k,$$

tức tổng vận tốc rơi của mọi quả trứng nằm ngoài đoạn $i..j$. Khi đó dễ thu được phương trình

$$\begin{aligned} f_1[i][j] &= y_i + \max\{f_1[i+1][j] - (x_{i+1} - x_i)w[i+1][j], \\ &\quad f_2[i+1][j] - (x_j - x_i)w[i+1][j]\}, \\ f_2[i][j] &= y_j + \max\{f_2[i][j-1] - (x_j - x_{j-1})w[i][j-1], \\ &\quad f_1[i][j-1] - (x_j - x_i)w[i][j-1]\}. \end{aligned}$$

Đáp án là

$$\frac{\max\{f_1[1][n], f_2[1][n]\}}{1000}.$$

Độ phức tạp thời gian là $\mathcal{O}(n^2)$. Đến đây bài toán được giải quyết hoàn toàn.

Nhìn lại quá trình trên, điểm số khi bắn quả trứng đã được tách thành hai phần để tính. Việc toàn bộ quả trứng rơi xuống chỉ phụ thuộc vào thời gian di chuyển hiện tại, nên ta có thể tính trước điểm tương lai vào trạng thái hiện tại.

2.2 Bài toán 2: Bài toán sắp xếp thứ hạng (BOI 2007, ngày 1)

Đề bài

Biết điểm số của các thí sinh, nhiệm vụ là đưa ra thứ hạng của họ theo thứ tự điểm từ cao xuống thấp. Đáng tiếc, cấu trúc dữ liệu lưu thông tin thí sinh chỉ hỗ trợ một thao tác: chuyển một thí sinh từ vị trí i đến vị trí j . Thao tác này không làm thay đổi thứ tự tương đối của các thí sinh khác. Nếu $i > j$, các thí sinh ở vị trí từ j đến $i-1$ đều lùi về sau một vị trí; nếu $i < j$, các thí sinh ở vị trí từ $i+1$ đến j đều tiến lên trước một vị trí. Chi phí của thao tác chuyển là $i+j$, trong đó vị trí được đánh số từ 1.

Hãy xác định một dãy thao tác để sắp xếp thí sinh theo điểm giảm dần, sao cho tổng chi phí của toàn bộ quá trình nhỏ nhất.

Dữ liệu vào

Tệp `sorting.in`: dòng đầu là số nguyên n ($2 \leq n \leq 1000$), số thí sinh. Tiếp theo là n dòng, mỗi dòng chứa một số nguyên không âm S_i ($0 \leq S_i \leq 1000000$), điểm của một thí sinh. Có thể giả sử điểm của mọi người đôi một khác nhau.

Dữ liệu ra

Tệp `sorting.out`: dòng đầu là số nguyên chỉ số lần chuyển. Mỗi dòng tiếp theo mô tả một thao tác bằng hai số nguyên i, j , nghĩa là chuyển thí sinh ở vị trí i đến vị trí j .

Ví dụ

<code>sorting.in</code>	<code>sorting.out</code>
5	2
20	2 1
30	3 5
5	
15	
10	

Phân tích

Để tiện thảo luận, đổi giá trị của mỗi phần tử thành vị trí mục tiêu của nó; khi đó trạng thái đích là dãy $1, 2, \dots, n$. Giả sử có thêm người thứ $n + 1$ đang ở vị trí $n + 1$. Thoạt nhìn bài toán không có chỗ để bắt đầu, nhưng phân tích thêm một chút sẽ thấy: chuyển i đến vị trí j có hai cách nhìn, là chuyển chủ động hoặc chuyển bị động; trực giác cũng cho thấy nếu một người phải di chuyển nhiều lần thì không bằng đi thẳng đến nơi.

Giả thuyết 2.1. Mỗi người nhiều nhất di chuyển một lần. Nếu di chuyển thì chắc chắn di chuyển đến ngay trước người có số hiệu lớn hơn mình 1.

Vì chuyển một người ra sau sẽ làm vị trí của một số người giảm 1, từ đó giảm chi phí di chuyển tương lai của họ, nên người có số hiệu lớn hơn nên được xử lý trước.

Giả thuyết 2.2. Thực hiện thao tác theo thứ tự số hiệu giảm dần.

Chứng minh hai giả thuyết trên không phải trọng tâm của bài viết này, nên được lược bớt ở đây; chứng minh chi tiết nằm trong phụ lục.

Kết luận 2.1. Thao tác theo thứ tự số hiệu giảm dần. Với mỗi người x có hai lựa chọn:

1. Chuyển đến vị trí ngay trước $x + 1$.
2. Nếu x đang ở trước $x + 1$, không di chuyển; về sau sẽ chuyển mọi người nằm giữa x và $x + 1$ đến trước x .

Xét việc chuyển x đang ở vị trí i ra sau đến trước $x + 1$ đang ở vị trí j ($j > i$). Theo Kết luận 2.1, hiện tại các số nhỏ hơn $x + 1$ đều chưa di chuyển; nói cách khác, thứ tự tương đối của mọi số không lớn hơn x giống ban đầu. Đây là Hệ quả 2.1. Mặt khác, mọi số lớn hơn $x + 1$ hoặc đã được chuyển trước đó, hoặc $x + 1$ đã được chuyển đến trước chúng; vì thế mọi số lớn hơn $x + 1$ đều nằm sau $x + 1$. Đây là Hệ quả 2.2.

Cần chú ý rằng thao tác chuyển làm thay đổi vị trí, nên i và j không còn là vị trí ban đầu của x và $x + 1$. Ta phải làm gì? Do thứ tự tương đối không đổi, ta có thể khai thác chính điều này để suy ra vị trí tuyệt đối.

Ví dụ với dãy ban đầu 3, 2, 5, 1, 4, nếu đặt 5 ra sau 4 thì 1 và 4 đều tiến lên một vị trí. Nếu thay đổi cách nhìn, cho 5 và 4 cùng dùng vị trí ban đầu 5, thì 1 và 4 xem như không di chuyển, trong khi thứ tự tương đối của 1, 2, 3, 4 vẫn giữ nguyên. Thực ra ta có thể dùng thứ tự tương đối để tính vị trí tuyệt đối: nếu vị trí ban đầu của x là p_1 , theo Hệ quả 2.1 và 2.2, vị trí thực của x bằng số lượng phần tử nhỏ hơn x trong đoạn ban đầu $1..p_1$, cộng thêm 1 cho chính x . Nhưng $x + 1$ có thể đã di chuyển, nên vị trí của nó trong dãy ban đầu cần được liệt kê; gọi vị trí đó là p_2 .

Ký hiệu $c(p, x)$ là vị trí thực của phần tử x sau khi đã xử lý xong các phần tử $x + 1, \dots, n$, nếu x đang ứng với vị trí p trong dãy ban đầu. Khi đó $c(p, x)$ bằng số lượng phần tử nhỏ hơn x trong đoạn ban đầu $1..p$, cộng thêm 1. Chi phí chuyển x từ vị trí ban đầu p_1 đến trước $x + 1$ ở vị trí ban đầu p_2 là

$$c(p_1, x) + c(p_2, x + 1) - 1,$$

vì ta chèn x vào trước $x + 1$ nên vị trí đích phải trừ 1.

Chẳng hạn, trong dãy 3, 2, 5, 1, 4, chuyển 3 đến trước 4. Từ hình minh họa trong bài gốc có thể thấy vị trí thực xuất phát là $i = 1$, vị trí đích là $j = 4$. Vị trí thực của 3 là số phần tử nhỏ hơn 3 đứng trước 3 trong dãy ban đầu, cộng 1, tức 1; tương tự, vị trí thực của 4 là 4 vì 5 chắc chắn ở sau 4. Chi phí lần chuyển này là $1 + 4 - 1$, khớp với tình hình thực tế. Để không ảnh hưởng đến vị trí của 1 và 2, sau khi chuyển, vị trí trong dãy ban đầu của 3 trở thành 5; nghĩa là sau lần chuyển này, các phần tử 3, 4, 5 đều ứng với vị trí ban đầu 5.

Đặt $f[x][p_2]$ là chi phí nhỏ nhất khi chuyển x đến vị trí p_2 trong dãy ban đầu, và các phần tử $x + 1, \dots, n$ đều đang ở vị trí p_2 . Gọi $pos[x]$ là vị trí ban đầu của x . Với trường hợp chuyển ra sau, ta có

$$f[x][p_2] = f[x + 1][p_2] + c(pos[x], x) + c(p_2, x + 1) - 1, \quad p_2 > pos[x].$$

Tiếp theo xét trường hợp vị trí đích ở trước vị trí xuất phát. Nếu ban đầu 4 và 5 đều không di chuyển, khi chuyển 3 đến trước 4, vị trí của 3 không phải $c(5, 3)$ mà là $c(5, 3) + 2$. Nếu trước đó quyết định của 5 là chuyển ra cuối, còn 4 không di chuyển, tức 4 vẫn ở vị trí ban đầu 2, thì khi chuyển 3 đến trước 4, vị trí của 3 là $c(5, 3) + 1$. Nói cách khác, quyết định trong quá khứ ảnh hưởng đến chi phí di chuyển hiện tại của 3. Ta mô phỏng cách xử lý ở bài toán 1: tính ảnh hưởng lên tương lai vào trạng thái hiện tại. Khi đó trường hợp $j < i$ có thể quy về trường hợp $j > i$.

Ta cần tính ảnh hưởng lên trạng thái tương lai. Nếu x không di chuyển, thì $x + 1$ chắc chắn ở một vị trí nào đó phía sau x ; gọi vị trí trong dãy ban đầu của chúng lần lượt là p_1, p_2 , trong đó $p_1 = pos[x]$. Khi quyết định 5 không di chuyển, nếu 4 chuyển đến trước 5, hoặc 4 cũng không di chuyển, thì 3 đều phải vượt qua cả 4 và 5. Nghĩa là trong lần di chuyển tương lai, vị trí của 3 là $c(5, 3) + (5 - 3)$. Phần $c(5, 3)$ sẽ được tính ở lần di chuyển tương lai, còn hiện tại chỉ cần cộng thêm phần chắc chắn phát sinh là $5 - 3$.

Tóm lại, nếu quyết định x không di chuyển, tuy bước này không sinh chi phí trực tiếp, ta vẫn phải tính trước một phần chi phí chắc chắn sẽ xuất hiện trong tương lai: tổng $x - s[k]$ trên các số nhỏ hơn x nằm sau $pos[x]$ và trước p_2 . Viết thành phương trình:

$$f[x][pos[x]] = \min_{p_2 > pos[x]} \left\{ f[x + 1][p_2] + \sum_{\substack{k=pos[x]+1 \\ s[k] < x}}^{p_2-1} (x - s[k]) \right\}.$$

Tổng hợp lại, phương trình của bài này là

$$f[i][j] = f[i + 1][j] + c(pos[i], i) + c(j, i + 1) - 1, \quad j > pos[i],$$

và

$$f[i][pos[i]] = \min \left\{ f[i][pos[i]], f[i + 1][j] + \sum_{\substack{k=pos[i]+1 \\ s[k] < i}}^{j-1} (i - s[k]) \right\}, \quad j > pos[i].$$

Đáp án là $\min_{1 \leq i \leq n} f[1][i]$. Độ phức tạp thời gian là $\mathcal{O}(n^2)$.

Tiểu kết

Nhìn lại quá trình giải bài toán 1, trước hết ta phát hiện chi phí của “hành động” hiện tại chịu ảnh hưởng của các quyết định quá khứ. Nếu thêm trạng thái t để biểu diễn ảnh hưởng quá khứ thì số trạng thái quá lớn, nên ta tính ảnh hưởng đó ngay tại thời điểm quyết định quá khứ và truyền nó qua trạng thái.

Làm như vậy cần hai điều kiện. Thứ nhất, ảnh hưởng phải là tất yếu. Trong bài toán 1, chỉ cần ta di chuyển thì quả trứng sẽ rơi. Trong bài toán 2, chỉ cần ta quyết định không di chuyển, chi phí tương lai sẽ tăng một giá trị cố định. Nếu bài toán 1 không yêu cầu bắn mọi quả trứng, những quả không bắn trong tương lai sẽ không chịu ảnh hưởng, và phương pháp trên không còn dùng được.

Thứ hai, ảnh hưởng phải là một hàm của quyết định hiện tại. Ở bài toán 1, hàm là $-t \sum v$; trong đó $\sum v$ liên quan đến trạng thái và có thể xem như hằng số, còn $-t$ chính là thời gian di chuyển của quyết định hiện tại. Bài toán 2 cũng thỏa mãn tính chất này.

Hai điều kiện trên gộp lại có nghĩa là: quyết định hiện tại ảnh hưởng đến chi phí của “hành động” tương lai chỉ thông qua chính quyết định hiện tại.

3 Ảnh hưởng còn phụ thuộc vào tình huống tương lai

Một số bài toán không đơn giản như trên. Ảnh hưởng của quyết định hiện tại lên chi phí “hành động” tương lai còn có thể phụ thuộc vào tình huống tương lai. Khi đó ta thường mở thêm vài chiều trạng thái để giả định quyết định tương lai, từ đó vẫn tính trước được một phần chi phí như trước; đến khi đi tới quyết định tương lai thì dùng trực tiếp trạng thái đã giả định.

3.1 Bài toán 3: Xóa khối (Từ Nghệ thuật thuật toán và thi tin học)

Đề bài

Jimmy vừa mua một chiếc máy tính, và giống phần lớn trẻ em, cậu nhanh chóng mê trò chơi. Gần đây Jimmy đang chơi trò xóa khối. Luật chơi như sau: có n ô màu xếp thành một hàng. Các khối cùng màu liền kề tạo thành một vùng; nếu hai khối kề nhau có cùng màu thì chúng thuộc cùng một vùng. Người chơi có thể chọn tùy ý một vùng để xóa. Nếu vùng đó chứa x khối, người chơi được x^2 điểm. Sau khi một vùng bị xóa, các khối ở bên phải dịch sang trái và nối với phần bên trái.

Phân tích

Trước hết gộp các khối cùng màu liên tiếp. Dùng cặp (a, b) để biểu thị b khối màu a nối liền nhau.

Mỗi lần xóa một vùng và cộng điểm của vùng đó, nhìn rất giống bài toán gộp đá. Ta dễ thử đặt $f[i][j]$ là điểm lớn nhất khi xóa hết các vùng từ i đến j . Xét vùng j : nếu xóa riêng nó, thì

$$f[i][j] = f[i][j-1] + b_j^2.$$

Còn một khả năng khác: vùng j hợp nhất với một số vùng cùng màu trước đó rồi mới bị xóa. Nhưng để tính điểm cần biết độ dài sau khi hợp nhất, điều không thể biết chỉ từ trạng thái hiện tại. Nếu thêm trạng thái để giả định tình huống quá khứ, ta phải biểu diễn từng khối trong quá khứ đã bị xóa hay chưa, số trạng thái tăng đến mức không thể chịu được.

Giả sử vẫn muốn dùng cách của hai bài trước: hiện tại xóa một vùng độ dài 3, trong quá khứ để lại một vùng độ dài 3, tổng điểm khi xóa chung là $6^2 = 36$. Nhưng 36 không thể tách ra để tính rải rác như ở hai bài trên. Nguyên nhân là hàm điểm là hàm bậc hai; khi quá khứ để lại một vùng độ dài 3, ảnh hưởng của nó lên việc xóa chung hiện tại còn phụ thuộc vào độ dài vùng hiện tại.

Vì vậy cần đổi cách nghĩ. Ta muốn hợp nhất vùng j với một số vùng cùng màu trước nó. Nếu lần hợp nhất này đã được dự liệu từ trước, điểm số đã được tính trước và truyền đến hiện tại qua trạng thái, thì ta có thể xây dựng phương trình quy hoạch động. Nói cách khác, với $f[i][j]$ hiện tại ta còn phải giả định rằng sau một số thao tác xóa, sẽ có k khối cùng màu với a_j nối vào sau vùng j , tạo thành một vùng mới dài $b_j + k$. Đặt $f[i][j][k]$ là điểm lớn nhất khi xóa/gộp đoạn từ i đến j , với giả định tương lai sẽ có k khối cùng màu a_j nối với j .

Vì sao chỉ cần ghi lại tương lai của j , mà không cần ghi lại tương lai của các vùng từ i đến $j - 1$?

Chứng minh 3.1. Nếu vùng j trong tương lai sẽ nối với một vùng k_2 , thì các vùng từ $j + 1$ đến $k_2 - 1$ cần bị xóa trước vùng j . Vì thế các vùng trước j chắc chắn không thể nối với các vùng nằm giữa j và k_2 . Nếu j trong tương lai nối với k_1 , còn một điểm i trước j nối với k_2 , thì ta có thể tính điểm xóa i trong khi tính trạng thái $f[i][k_2]$.

Do đó có **Kết luận 3.1**: với trạng thái $f[i][j][k]$, bất kể thực hiện thao tác nào, các vùng từ i đến $j - 1$ chỉ có thể nối với các vùng trong đoạn $i..j$, còn vùng j có thể nối với các vùng phía sau j .

Từ đó thu được phương trình:

1. Nếu xóa trực tiếp vùng thứ j cùng với k khối tương lai nối vào sau nó, thì

$$f[i][j][k] = f[i][j - 1][0] + (b_j + k)^2.$$

2. Nếu j hợp nhất với một vùng trước đó, giả sử vùng cùng màu với j là p , thì

$$f[i][j][k] = f[i][p][k + b_j] + f[p + 1][j - 1][0], \quad i \leq p < j, \quad a_p = a_j.$$

Lấy giá trị lớn hơn trong hai loại chuyển trên. Đáp án là $f[1][n][0]$. Độ phức tạp thời gian là $\mathcal{O}(n^4)$.

Ban đầu ta muốn phân bổ chi phí vào từng quyết định “giữ lại để sau này xóa”, nhưng phát hiện các chi phí này không độc lập mà liên hệ với nhau: chi phí hiện tại còn cần biết tình huống tương lai. Vì vậy ta mở thêm một chiều trạng thái để ghi lại tình huống tương lai.

3.2 Bài toán 4: Logistics Olympic (NOI 2008, ngày 2)

Đề bài

Olympic Bắc Kinh 2008 sắp khai mạc, cả nước đang chuẩn bị cho sự kiện trọng đại này. Để tổ chức Olympic hiệu quả và thành công, việc quy hoạch hệ thống logistics là không thể thiếu. Hệ thống logistics gồm một số trạm cơ sở, đánh số từ 1 đến N . Mỗi trạm i có đúng một trạm kế tiếp S_i , và có thể có nhiều trạm tiền nhiệm. Vật tư cần vận chuyển tiếp ở trạm i sẽ được đưa đến trạm kế tiếp S_i . Hiển nhiên trạm kế tiếp của một trạm không thể là chính nó. Trạm số 1 gọi là trạm điều khiển; từ bất kỳ trạm nào cũng có thể vận chuyển vật tư đến trạm điều khiển. Chú ý rằng trạm điều khiển cũng có trạm kế tiếp, để khi cần có thể lưu thông vật tư.

Trong hệ thống logistics, độ tin cậy cao và chi phí thấp là hai mục tiêu thiết kế chính. Với trạm i , định nghĩa độ tin cậy $R(i)$ như sau. Giả sử trạm i có w trạm tiền nhiệm $P_1, P_2, \dots, P_w$, tức các trạm này chọn i làm trạm kế tiếp. Khi đó

$$R(i) = C_i + k \sum_{j=1}^w R(P_j),$$

trong đó C_i và k là các hằng số thực dương, và $k < 1$. Độ tin cậy của cả hệ thống tương quan dương với độ tin cậy của trạm điều khiển. Mục tiêu là sửa đổi hệ thống, tức thay đổi trạm kế tiếp của một số trạm, để $R(1)$ lớn nhất có thể. Do hạn chế kinh phí, nhiều nhất chỉ được sửa trạm kế tiếp của m trạm, và không được sửa trạm kế tiếp của trạm điều khiển. Bài toán là: thay đổi không quá m trạm kế tiếp sao cho $R(1)$ lớn nhất.

Dữ liệu vào

Tệp `trans.in`: dòng đầu gồm hai số nguyên và một số thực N, m, k , trong đó N là số trạm, m là số trạm kế tiếp được sửa nhiều nhất, còn k là hằng số trong định nghĩa độ tin cậy. Dòng thứ hai gồm N số nguyên $S_1, S_2, \dots, S_N$. Dòng thứ ba gồm N số thực dương $C_1, C_2, \dots, C_N$.

Dữ liệu ra

Tệp `trans.out`: in một số thực duy nhất, giá trị lớn nhất của $R(1)$, chính xác đến hai chữ số sau dấu thập phân.

Ví dụ

```
4 1 0.5
2 3 1 3
10.0 10.0 10.0 10.0
```

30.00

Phân tích

Nối mỗi điểm đến trạm kế tiếp của nó bằng một cạnh. Phân tích đơn giản cho thấy đồ thị của bài là một số cây có gốc nằm trên một chu trình; các gốc cây nối với nhau thành chu trình đó.

Với một cây gốc i , lặp lại phương trình định nghĩa sẽ thấy $R(i)$ bằng tổng

$$\sum_x C_x k^{d(x,i)}$$

trên mọi điểm x trong cây gốc i , trong đó $d(x, i)$ là số cạnh trên đường từ x đến i .

Xét chu trình. Với một chu trình độ dài 3, chẳng hạn

$$R(1) = C_1 + kR(2), \quad R(2) = C_2 + kR(3), \quad R(3) = C_3 + kR(1).$$

Khử biến được

$$R(1) = C_1 + kC_2 + k^2C_3 + k^3R(1),$$

nên

$$R(1) = \frac{C_1 + kC_2 + k^2C_3}{1 - k^3}.$$

Tức là tổng đóng góp $C_x k^{d(x,1)}$ của mỗi điểm x trên chu trình được chia cho $1 - k^{len}$, trong đó len là độ dài chu trình.

Với một điểm x nằm trong cây treo vào điểm i trên chu trình, đóng góp của nó vào $R(1)$ là

$$C_x k^{d(x,i)} k^{d(i,1)} = C_x k^{d(x,1)}.$$

Vì thế

$$R(1) = \frac{\sum_{i=1}^n C_i k^{d(i,1)}}{1 - k^{len}}.$$

Ta liệt kê độ dài chu trình, tức liệt kê cách sửa trạm kế tiếp của một điểm trên chu trình. Khi đó mẫu số $1 - k^{len}$ đã xác định, chỉ cần làm cho tử số lớn nhất có thể. Vì chu trình đã xác định, ta không được sửa tiếp trạm kế tiếp của gốc cây; những sửa đổi còn lại chỉ có thể đổi trạm kế tiếp của một điểm không phải

gốc cây. Theo công thức trên, mỗi lần sửa như vậy hiển nhiên nên đặt trực tiếp trạm kế tiếp của điểm được sửa thành 1.

Thử dùng quy hoạch động. Với cây gốc i , rất tự nhiên đặt trạng thái là giá trị lớn nhất trong cây gốc i khi đã sửa j lần. Với điểm i có hai lựa chọn: sửa hoặc không sửa.

Nếu không sửa trạm kế tiếp của i , ta phân phối j lần sửa cho các cây con của i , rồi cộng đóng góp của i trong trạng thái hiện tại: $C_i k^{d(i,1)}$.

Nếu sửa trạm kế tiếp của i thành 1, đóng góp của cả cây gốc i vào $R(1)$ phải tính thế nào? Nếu con cháu của i đều chưa từng sửa, đặt trạm kế tiếp của i thành 1 sẽ làm khoảng cách từ i và mọi con cháu của i đến 1 giảm 2, tức đóng góp đều chia cho k^2 . Nhưng nếu trạm kế tiếp của một điểm con trước đó đã được đặt thành 1, thì khi ta sửa trạm kế tiếp của i thành 1, khoảng cách của điểm con đó không hề giảm 2. Nói cách khác, quyết định ở điểm con ảnh hưởng đến chi phí khi sửa điểm i .

Liệu có thể thêm trạng thái để biểu diễn trong cây i những điểm nào đã sửa, những điểm nào chưa sửa? Rõ ràng số trạng thái là không thể chấp nhận. Ta cần tính đóng góp của điểm con vào chi phí sửa điểm i ngay khi quyết định ở điểm con đó, tức khi quyết định điểm con, cộng luôn chi phí tương lai. Nhưng phải cộng bao nhiêu? Nếu trạm kế tiếp của i được sửa thành 1, khoảng cách của điểm con là 2. Nếu i không sửa mà một điểm tổ tiên y sửa thành 1, thì khoảng cách của điểm con là 3. Đóng góp của điểm con phụ thuộc vào tổ tiên được sửa gần nó nhất, vì vậy cần thêm trạng thái biểu diễn tổ tiên được sửa gần nhất.

Đặt $f[i][j][d]$ là giá trị lớn nhất trong cây gốc i , khi sửa j lần, và khoảng cách từ i đến 1 là d . Xét một con s của i : khoảng cách của s hoặc là 1 nếu sửa trạm kế tiếp của s , hoặc là $d(i) + 1$ nếu không sửa. Đặt

$$g[i][j][d] = \max\{f[i][j][d+1], f[i][j][1]\}.$$

Khi đó có các phương trình sau.

1. Nếu i không sửa trạm kế tiếp, thì trừ trường hợp bản thân i thật sự có khoảng cách 1 đến 1, ta không thể có $d = 1$. Giả sử các con của i là $s_1, s_2, \dots, s_t$, khi đó

$$f[i][j][d] = \max_{j_1 + \dots + j_t = j} \left\{ \sum_{r=1}^t g[s_r][j_r][d] \right\} + C_i k^d.$$

Phần lấy cực đại có thể thực hiện thêm một lần quy hoạch động kiểu ba lô. Gọi $FF[q][u]$ là giá trị lớn nhất khi đã phân phối u lần sửa cho q người con đầu tiên, ta có

$$FF[q][u] = \max_{0 \leq v \leq u} \{FF[q-1][v] + g[s_q][u-v][d]\}.$$

Giá trị cực đại cần lấy là $FF[t][j]$.

2. Nếu sửa trạm kế tiếp của i , thì

$$f[i][j][1] = \max_{j_1 + \dots + j_t = j-1} \left\{ \sum_{r=1}^t g[s_r][j_r][1] \right\} + C_i k.$$

Cuối cùng xét chu trình. Vì chu trình đã được liệt kê và xác định, đáp án cho phần tử số là

$$\max_{j_1 + \dots + j_{len} = m} \{f[p_1][j_1][0] + f[p_2][j_2][1] + \dots + f[p_{len}][j_{len}][len-1]\},$$

trong đó $p_1, p_2, \dots, p_{len}$ là các điểm trên chu trình. Ví dụ với chu trình độ dài 3, đáp án là

$$\max_{j_1 + j_2 + j_3 = m} \{f[1][j_1][0] + f[2][j_2][1] + f[3][j_3][2]\}.$$

Phần này cũng có thể giải bằng một quy hoạch động nữa. Vì mỗi điểm chỉ bị tính ba lô một lần, độ phức tạp thời gian là $\mathcal{O}(len \cdot n^2 m^2)$, đủ để vượt qua giới hạn thời gian.

Những năm gần đây xuất hiện nhiều bài tương tự, như IOI 2005 “Rivers” và NOI 2006 “Network toll”. Các bài này đều quy hoạch động trên cây, đồng thời chuyển phần chi phí vốn nên tính ở điểm i xuống các hậu duệ của i . Khi quy hoạch các hậu duệ, ta giả định tình huống tương lai và ghi lại bằng trạng thái; trong bài này, đó là giả định vị trí điểm sửa gần nhất, để khi quy hoạch đến i có thể trực tiếp dùng quyết định đã giả định trong quá khứ.

Tiểu kết

Loại bài này có thể xem là phiên bản phức tạp của loại thứ nhất. Ta vẫn phát hiện quyết định quá khứ ảnh hưởng đến chi phí “hành động” hiện tại; ta vẫn phát hiện nếu ghi lại quyết định quá khứ trong trạng thái hiện tại thì số trạng thái quá lớn; vì vậy phải tính trước phần ảnh hưởng đó trong quá khứ. Tuy nhiên ảnh hưởng của quyết định hiện tại lên chi phí “hành động” tương lai còn liên quan đến tình huống tương lai. Khi đó ta giả định tình huống tương lai, rồi truyền các ảnh hưởng khác nhau đến tương lai theo các trạng thái khác nhau.

4 Tổng kết

Bốn ví dụ trong bài đều có phương trình không dễ xây dựng. Nguyên nhân là chi phí của “hành động” hiện tại chịu ảnh hưởng của các quyết định quá khứ; còn nếu thêm trạng thái để giả định quá khứ ở hiện tại thì số trạng thái quá lớn. Vì vậy ta thay đổi “quan niệm thời gian”: thay vì nhìn hiện tại từ quá khứ, ta nhìn tương lai từ hiện tại, và tính ảnh hưởng của quyết định hiện tại lên tương lai vào trạng thái hiện tại.

Với loại bài thứ nhất, chỉ cần xem ảnh hưởng lên tương lai là chi phí của quyết định hiện tại và lưu trong trạng thái hiện tại. Với loại bài thứ hai, cần giả định tình huống tương lai, lưu ảnh hưởng của các tình huống tương lai khác nhau trong các trạng thái khác nhau. Có thể hiểu là truyền ảnh hưởng đến tương lai theo những con đường khác nhau, để khi đến quyết định tương lai thì dùng trực tiếp.

Dĩ nhiên hai loại bài này có biến hóa vô tận. Nhưng chỉ cần giải phát hiện và dám đổi mới, ta sẽ xây dựng được phương trình.

Lời cảm ơn

Xin chân thành cảm ơn các thầy Cao Lợi Quốc và Chu Tổ Tùng đã hướng dẫn, giúp đỡ tôi khi viết bài này. Xin chân thành cảm ơn các bạn Tầm Vân Ba, Lý Viễn Thao, Ngô Không, Dịch Thiến, Đặng Phương Đình, Ngô Nhất Phàm và nhiều bạn khác đã giúp đỡ tôi.

Tài liệu tham khảo

[A] *Nghệ thuật thuật toán và thi tin học*, Lưu Nhữ Giai và Hoàng Lượng, Nhà xuất bản Đại học Thanh Hoa.

[B] Bài giảng WC 2008 của Tôn Huy.

[C] Lời giải chính thức BOI 2007.

A Chứng minh cho Giả thuyết 2.1 và 2.2

Phần chứng minh sau lấy từ lời giải chính thức BOI 2007.

Xét thí sinh $p_{\min}$ có điểm nhỏ nhất; cuối cùng người này phải đứng ở vị trí n .

Mệnh đề 1. Nếu $p_{\min}$ được di chuyển, phương án tối ưu là chuyển thẳng người này đến cuối danh sách.

Chứng minh. Hiển nhiên chuyển $p_{\min}$ về phía trước chỉ làm tăng chi phí của những người khác cần di chuyển. Giả sử người này được chuyển ra sau đến một vị trí $i < n$, tức tiết kiệm $n - i$ bước so với chuyển đến cuối. Khi đó có hai khả năng: hoặc $n - i$ người phía sau người này trong danh sách phải được di chuyển, và chi phí của họ đều tăng thêm 1 so với trường hợp $p_{\min}$ đã được chuyển đến cuối; hoặc $p_{\min}$ phải di chuyển thêm lần nữa, điều hiển nhiên không tối ưu.

Mệnh đề 2. Nếu $p_{\min}$ được di chuyển, ta có thể thực hiện việc này ở bước đầu tiên.

Chứng minh. Từ Mệnh đề 1, ta đã biết phần thứ hai của chi phí di chuyển là cố định. Vì vậy lý do duy nhất để không chuyển $p_{\min}$ trước tiên là vị trí hiện tại của người này có thể giảm do các thí sinh khác được chuyển đi. Nhưng chi phí di chuyển của những thí sinh đó lại tăng thêm 1 vì $p_{\min}$ chưa được chuyển đến cuối.

Nếu $p_{\min}$ được di chuyển, bài toán giảm thành cùng một bài toán với $n - 1$ thí sinh. Nếu $p_{\min}$ không di chuyển, ta có một bài toán tương tự với $n - 1$ thí sinh, kèm ràng buộc bổ sung rằng mọi thí sinh đứng sau $p_{\min}$ phải được di chuyển trước $p_{\min}$.

Gọi $p'_{\min}$ là thí sinh có điểm nhỏ nhất trong $n - 1$ thí sinh còn lại. Ta thấy không có ích gì khi chuyển $p'_{\min}$ ra sau $p_{\min}$, vì như vậy sẽ phải chuyển người này thêm lần nữa, và việc vượt qua các thí sinh vẫn chưa di chuyển cũng không đem lại lợi ích: với mỗi thí sinh như vậy, ta giảm chi phí di chuyển của họ đi 1, nhưng lại tăng chi phí di chuyển của $p'_{\min}$ ít nhất 1. Nếu $p'_{\min}$ hiện đứng trước $p_{\min}$ trong danh sách, bằng lập luận tương tự Mệnh đề 1, nếu $p'_{\min}$ được di chuyển thì tối ưu là chuyển thẳng đến trước $p_{\min}$. Mệnh đề 2 vẫn còn đúng.

Trường hợp còn lại là $p'_{\min}$ hiện đứng sau $p_{\min}$ và phải được chuyển về phía trước. Nếu có các thí sinh nằm giữa $p_{\min}$ và $p'_{\min}$, ta có thể cân nhắc chuyển họ trước. Nếu chuyển $p'_{\min}$ trước, vị trí xuất phát của các thí sinh ở giữa tăng thêm 1. Nhưng nếu chuyển một thí sinh ở giữa trước, vị trí đích mà $p'_{\min}$ cần chuyển đến cũng tăng thêm 1. Do đó chuyển $p'_{\min}$ trước không thể tệ hơn.

Vì vậy, bằng quy nạp và các lập luận trên, ta chứng minh được rằng mỗi thí sinh được di chuyển nhiều nhất một lần, và các thí sinh có thể được di chuyển theo thứ tự điểm tăng dần, tức theo số hiệu mục tiêu giảm dần trong cách đánh số của phần phân tích.

B Mã nguồn

Phụ lục B của bản PDF gốc chỉ hiện các tiêu đề “mã nguồn bài toán 1”, “mã nguồn bài toán 2” và “mã nguồn bài toán 4” trong kết quả trích xuất văn bản; nội dung chương trình không được pdftotext trích xuất thành văn bản có thể dịch.